

U01 · Introducción a la programación

Antes de escribir una sola línea de Java vamos a entender **qué es realmente programar**. Esta unidad es la base de todo el módulo: si la dominas, el resto del curso te costará mucho menos.

1. ¿Qué es un programa?

La RAE define un programa como un “*conjunto de instrucciones que permite a un ordenador realizar funciones diversas*”. Dicho de otra forma más útil: un programa es una **receta** que un ordenador sigue al pie de la letra, sin criterio propio — hace exactamente lo que le dices, ni más ni menos (y ese es justo el motivo por el que los errores de programación existen).

Tres términos que se confunden mucho y que conviene distinguir desde el minuto uno:

- **Programa:** un conjunto de instrucciones ejecutables por un dispositivo.
- **Proceso:** un programa *en ejecución*, en un instante concreto (con su propio estado: qué instrucción está ejecutando, qué valores tienen sus variables, qué ficheros tiene abiertos...).
- **Aplicación:** uno o más programas junto con su documentación y los ficheros que necesita para funcionar — el conjunto que el usuario final percibe como “una herramienta”.

La caja negra

Una forma útil de pensar en cualquier programa, sin importar lo complejo que sea por dentro: imagina que es una **caja negra**. No te importa cómo funciona por dentro, solo te importa que **lee** información del exterior (teclado, fichero, red, sensor...), la **transforma** y **escribe** un resultado (pantalla, fichero, red...). Vamos a pasarnos el curso abriendo esa caja negra, pieza a pieza.

2. Lenguajes de programación

Existen más de mil lenguajes de programación distintos. Para no perdernos, conviene clasificarlos por varios criterios:

- **Nivel de abstracción:** lenguajes de **bajo nivel** (ensamblador, muy cercano al hardware) frente a lenguajes de **alto nivel** (Java, Python... mucho más cercanos a cómo pensamos los humanos).
- **Tipado:** lenguajes **fuertemente tipados** (como Java, donde cada variable tiene un tipo fijo que el compilador vigila) frente a lenguajes de **tipado flexible** (como Python o JavaScript).
- **Paradigma:** la “filosofía” con la que se organiza el código — estructurado, orientado a objetos, funcional... Java es principalmente orientado a objetos, aunque desde hace años incorpora también características funcionales (lo verás en la unidad de la API Stream).

De tu código a algo que la máquina entiende

Un ordenador solo entiende **código máquina** (ceros y unos). Para llegar ahí desde el código que escribes hay, básicamente, dos caminos:

- **Compilación:** un programa (el **compilador**) traduce todo tu código fuente a código máquina *antes* de ejecutarlo, generando un fichero ejecutable. Es rápido en ejecución, pero hay que recompilar para cada plataforma (Windows, Linux, Mac...).
- **Interpretación:** un programa (el **intérprete**) va leyendo y ejecutando tu código línea a línea, en el momento. Es más lento, pero el mismo código fuente funciona en cualquier plataforma que tenga el intérprete instalado.

❗ ¿Y Java qué hace?

Java hace un poco de cada cosa, y es la razón de su famoso lema “*write once, run anywhere*”. Tu código `.java` se **compila** a un formato intermedio llamado **bytecode** (ficheros `.class`), que no es código máquina real sino instrucciones para una máquina virtual. Después, la **JVM (Java Virtual Machine)** —que sí es específica de cada sistema operativo— **interpreta** (o compila en caliente, con el JIT) ese bytecode. Por eso el mismo `.class` funciona igual en Windows, Linux o Mac: solo necesitas tener instalada la JVM correspondiente.

3. El ciclo de vida de un programa

Escribir código es solo una fase del proceso. Un desarrollo de software serio pasa por varias etapas:

1. **Análisis:** entender el problema y qué necesita el cliente/usuario. Es la fase que más se salta la gente sin experiencia, y la que más caro sale saltarse.
2. **Diseño:** decidir *cómo* se va a resolver el problema antes de escribir código — qué estructuras de datos, qué algoritmo, qué clases (en los próximos temas verás cómo dibujar esto con pseudocódigo y, más adelante, con diagramas UML).
3. **Implementación (codificación):** aquí es donde por fin escribimos el programa en un lenguaje concreto.
4. **Pruebas:** comprobar que el programa hace lo que se esperaba, con casos normales y casos límite.
5. **Despliegue y mantenimiento:** poner el programa en manos de quien lo va a usar, y corregirlo/mejorarlo mientras siga vivo (que suele ser la fase más larga de todas).

★ **Be the Code:** la mayoría del código que vas a mantener en tu vida profesional **no lo habrás escrito tú**. Documentar bien tus programas (comentarios claros, nombres de variable que se entiendan) no es un capricho estético: es pensar en la persona (a veces tú mismo, seis meses después) que va a tener que entender tu código sin preguntarte.

4. Algoritmos y pseudocódigo

Un **algoritmo** es una secuencia finita y ordenada de pasos que resuelve un problema. Antes de traducir un algoritmo a Java (unidad 2), vamos a aprender a expresarlo en **pseudocódigo**: una notación intermedia, en español y sin la sintaxis estricta de ningún lenguaje real, que nos deja pensar en la lógica sin pelearnos todavía con punto y coma ni llaves.

Variables y expresiones

Una variable es un espacio con nombre donde guardamos un valor que puede cambiar durante la ejecución. $A = B$ significa “copia el valor de B en A” — no es una igualdad matemática, es una **asignación**, y el orden importa muchísimo.

Las **expresiones lógicas** combinan comparaciones ($>$, $<$, $==$...) con operadores lógicos (AND, OR, NOT) y siempre se evalúan a Verdadero o Falso.

Sentencias alternativas (decidir)

```
Si (condición) Entonces
    // se ejecuta si la condición es verdadera
Sino
    // se ejecuta si es falsa
FinSi
```

Sentencias iterativas (repetir)

Pseudocódigo distingue varias formas de repetir un bloque de instrucciones, según si conoces de antemano cuántas veces se repite y cuándo se comprueba la condición:

- **Mientras (while):** comprueba la condición *antes* de cada vuelta — puede que no se ejecute ni una vez.
- **Repetir...hasta (do-while):** comprueba la condición *después* — se ejecuta siempre al menos una vez.
- **Para (for):** cuando ya sabes cuántas repeticiones necesitas (por ejemplo, “desde 1 hasta 10”).

? ¡No Hay Preguntas Tontas! ¿Cuál es la diferencia real entre Mientras y Repetir...hasta si al final las dos repiten cosas? La diferencia está en el orden: Mientras pregunta primero y actúa después (puede que no actúe nunca), y Repetir...hasta actúa primero y pregunta después (actúa siempre al menos una vez). Es la misma diferencia que hay entre “mientras tengas hambre, come” (a lo mejor no tienes hambre y no comes nada) y “come, y repite hasta que no tengas más hambre” (comes seguro, al menos una vez).

Traza de un algoritmo

Trazar un algoritmo es ejecutarlo a mano, apuntando en una tabla cómo cambia el valor de cada variable en cada paso. Es la herramienta más básica (y más infravalorada) para depurar: antes de sospechar del ordenador, sospecha de tu lógica, y la traza te lo demuestra sobre el papel.

Subalgoritmos y programación modular

Un **subalgoritmo** (lo que en Java llamaremos **método**, unidad 2 y 4) es un bloque de pseudocódigo con nombre propio que resuelve una subtarea concreta y se puede reutilizar. Dividir un problema grande en subalgoritmos más pequeños es la idea central de la **programación modular**: cada pieza se entiende, se prueba y se reutiliza por separado, en lugar de escribir un único bloque gigante de instrucciones.

El error más habitual al empezar

Cuando alguien empieza a programar, la tentación es escribirlo todo en un único bloque enorme. Aunque “funcione”, es mucho más difícil de leer, probar y corregir. Acostúmbrate desde ya a pensar en subalgoritmos pequeños, cada uno con una única responsabilidad clara.

RA y CE que cubre esta unidad

RA	Criterios de evaluación cubiertos
RA1. Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.	a) estructura de un programa · c) entornos integrados de desarrollo · i) comentarios en el código

 Boletín inicial

 Boletín intermedio

 Extras